Vision Document: Clay Earth Dataset Foundry

> **How to use this document (developer teams):** This is your context and scope for the build, not a rigid spec. Read the Executive Summary and "The Three Tasks" first, pick one task to drive end to end (start with the ADB agentic eval), and treat the "In Scope / Out of Scope" lists as your guardrails for a 2-day build. A short list of items to confirm with Clay at kickoff is at the end.

Executive Summary

The Earth Dataset Foundry is an open platform that helps public-interest practitioners turn free satellite imagery into usable, trustworthy datasets. The hardest and least-served part of that mission—and the focus of this hackathon—is **evaluating agentic geospatial workflows**: when an AI agent plans and runs a dataset build by calling geospatial tools, how do you observe what it did and judge whether it did it well? Clay has the ingredients (known tasks, input data, example outputs, and a large catalog of MCP servers) but has never had the capacity to connect them into an evaluation system. The hackathon MVP builds that system: an **observability + evaluation sandbox** where an LLM harness calls geospatial tools over MCP, the agent's activity is captured as traces, and those traces plus the task output are summarized and scored against a rubric or reference examples. The point to prove is that an agentic geospatial workflow can be **observed, summarized, and scored end to end**, on one or more of Clay's real tasks.

Business Context

Problem Statement

Organizations working on urgent public problems have more satellite data than ever but lack the practical datasets they need, and building those datasets increasingly means orchestrating AI agents that call many specialized geospatial tools. The bottleneck Clay faces is not running an agent—it is **knowing whether to trust one**. Clay has long lists of evaluations it has never had capacity to run, a new text-native Clay model that needs synthetic caption data it cannot bandwidth-test, and partners (e.g., the Asian Development Bank) who need custom evals to understand how to use AI to build datasets. Without a way to observe and score agentic workflows, every agent-built dataset is a black box.

Business Drivers

- **Evaluating agentic workflows is the #1 unmet need.** Clay has assembled tasks, input data, and tooling but lacks the observability + scoring layer to judge agentic runs.
- **Clay v2 depends on caption data nobody has tested.** The next, text-native Clay model is trained on synthetic captions of satellite imagery; Clay has no bandwidth to evaluate those caption workflows.
- **Partners need eval capacity Clay cannot supply.** ADB and others need custom evals for AI-assisted dataset building; Clay's eval backlog is real and growing.
- **The ingredients already exist.** Known tasks, public input data (NAIP, Vantor), Clay/LGND embeddings, and a catalog of geospatial MCP servers mean a team can focus on the evaluation loop rather than assembling raw materials.

Target Users and Stakeholders

User Type	Description	Primary Need
Clay model & data team	Build and improve the Clay model and Foundry datasets	A way to evaluate agentic caption/dataset-build workflows without hand-checking every run
Partner organizations (e.g., ADB)	Need custom evals for AI-assisted dataset building	A reusable harness that scores an agent against their tasks and rubric
Agent / harness developers	Build the agentic workflows under test	Observable traces and clear scores that show where a workflow succeeds or fails
Domain experts	Judge whether outputs are trustworthy	Summarized traces + output scored against a rubric or reference examples

Business Constraints

- Open and reproducible by default (Apache-2.0, consistent with Clay's existing releases).
- Trust and provenance are first-class: every scored run keeps its traces, inputs, and the rubric/reference it was judged against.
- Public or synthetic data only for the hackathon (NAIP, Vantor open data, LGND embeddings, synthetic prompts/captions, and openly licensed reference data such as Global Renewables Watch, USGS USPVDB/USWTDB and OpenStreetMap).
- Keep the model and the tracing tool behind abstractions so both are swappable.

Solution Concept and Terminology

This section grounds the hackathon in how Clay frames the problem and sets shared vocabulary for the teams.

Core Concepts

Term	Definition
Task	A known unit of work to evaluate an agent on. Three are available this hackathon (below); teams pick one or more.
LLM harness	The agent (on Amazon Bedrock, kept swappable) that performs a task by calling geospatial tools.
MCP tools	Geospatial capabilities the agent reaches via Model Context Protocol servers. Clay maintains ~99 and will share a free/priority shortlist.
Trace	The captured record of the agent's activity—tool calls, sequence, failures, cost—produced while it runs.
Observability sandbox	The component that captures and summarizes traces so a human (or a score) can judge the run.

Term	Definition
Scoring / rubric	How a run is judged: trace-derived workflow signals plus output quality vs. a rubric or reference examples.
Fixed caption task	A caption task on one image of one place at one date.
Change caption task	A caption task on two images of the same place at different dates. The caption describes what changed.
Position, counts, relationality	What each caption task tests: where objects are, how many there are, and how they sit relative to each other (for example, a road between two fields).
Caption rubric	The five criteria used to score caption output. See "Caption Generation: Tasks and Rubric".

Design Principles

- **Evaluating the agent is the product.** The deliverable is the observe-summarize-score loop, not a one-off dataset.
- **Known task + known input + a reference to score against.** Every evaluated task needs all three; the scoping session confirmed these for the first two tasks.
- **Keep the model and tracing tool swappable.** The team recommends a tracing/observability tool but must not hard-wire it.
- **Recorded-trace fallback.** If a live agent is flaky, replay recorded traces through the scoring sandbox so the demo always lands.

The Three Tasks (teams pick one or more)

Avery confirmed teams should show up able to work on one or more of three tasks, with progress on each.

1. **ADB agentic eval (recommended starting task).** Evaluate an agent (or set of agentic harnesses) on an ADB-style "how to use AI / build datasets" task. Needs: input data, the harness(es) under test, a scoring rubric, and prompts (teams bring synthetic prompts); the output is the source of traces gathered from observing the system. The eval scores the agentic workflow itself.
2. **Caption generation (for the text-native Clay v2).** Input high-resolution satellite imagery (NAIP) plus sample Vantor data; the agent develops training captions and enables retrieval (e.g., "find railroads"). Captions cover fixed tasks (one image) and change tasks (two dates of the same place), and each tests position, counts and relationality. Output is scored on the five-criterion caption rubric below. Clay has example captions, and evaluation datasets check whether captions support the retrieval tasks.

Caption Generation: Tasks and Rubric

Task types

Task type	Input	What the caption must get right
Fixed	One image of a place at one date	Position, counts and relationality of the objects in the image
Change	Two images of the same place at different dates	Changes in position, counts and relationality between the two dates

Rubric

Five criteria. Weighting is still to be decided, so report each score on its own until it is.

Criterion	Question it answers
Retrieval quality	Can a caption be retrieved from a text query alone, with no image, using any off-the-shelf text model?
Grounding (most important)	Does the caption match what is actually on the ground, including objects, attributes and counts?
Fidelity	Does an LLM judge rate the caption as an accurate account of the image overall?
Cost	What are throughput and time to output for the same images and prompt across VLMs, measured in GPU time and number or size of GPUs rather than token cost?
Label preparation time	How long does it take to prepare the labels?

Retrieval answer key (ADB suggestion)

ADB (Kinzah) reviewed both documents and suggested open data for the retrieval task. Teams could train on one region of Global Renewables Watch v1.0 or USGS USPVDB/USWTDB and test on another region that already has an LGND collection, using GRW, USGS or OpenStreetMap there as the answer key. Sources are listed under External Dependencies.

MVP Scope

MVP Objective

Build an observability + evaluation sandbox that runs an LLM harness against a known Clay task, lets the agent call geospatial tools via MCP, captures the agent's traces, and summarizes and scores both the workflow and the output against a rubric or reference examples—proving an agentic geospatial workflow can be evaluated end to end.

MVP Success Criteria

- ☐ An LLM harness performs a chosen task (ADB eval or caption generation) by calling geospatial tools via MCP over public/synthetic input data.
- ☐ The agent's activity is captured as **traces** in an observability sandbox and **summarized** into a human-readable view.
- ☐ The run is **scored** on two dimensions: workflow signals from the traces (tool-call correctness/sequence, failures, steps/cost) and output quality vs. a rubric or reference examples.
- ☐ For the caption task, output is scored on each of the five **caption rubric** criteria: retrieval quality, grounding, fidelity, cost and label preparation time.
- ☐ The sandbox produces an **evaluation report** for the run.
- ☐ The tracing/observability tool and the model are both **swappable** (behind an abstraction).
- ☐ If the live agent fails, **recorded traces** can be replayed through the scoring sandbox to still produce a report.

Features In Scope (MVP)

Feature	Description	Priority	Rationale for Inclusion
LLM harness over MCP tools	Agent on Bedrock performs the task by calling geospatial MCP tools	Must Have	The agentic workflow is what we evaluate
Trace capture	Record the agent's tool calls, sequence, failures, cost	Must Have	No evaluation without observability
Trace summarization	Summarize traces into a readable view of what the agent did	Must Have	Clay explicitly wants to "summarize traces from agentic activity"
Scoring against rubric/reference	Score workflow + output vs. a rubric (ADB) or the five-criterion caption rubric and example captions	Must Have	A score is the point of the eval
Evaluation report	Per-run report combining summary + scores	Must Have	The deliverable a practitioner/partner reads
Swappable tracing tool + model	Abstraction over the observability tool and the Bedrock model	Must Have	Confirmed requirement: team recommends, keep swappable
Recorded-trace replay (fallback)	Replay stored traces through scoring when the live agent is flaky	Must Have	Keeps the demo reliable

Feature	Description	Priority	Rationale for Inclusion
Caption cost comparison	Run the same images and prompt through more than one VLM and record GPU time, GPU count and size, and time to output	Should Have	The caption rubric compares VLMs on hardware use, not tokens
Second task	Run the loop on a second of the three tasks	Exploratory	Shows the eval generalizes if time allows
Multi-agent traces	Planner/executor/checker traces	Exploratory	Bolder architecture if time allows
Minimal UI	Thin view to launch a run and read the report	Should Have	Makes the loop demonstrable

Features Explicitly Out of Scope (MVP)

Feature	Reason for Deferral
Training the Clay v2 model itself	The hackathon evaluates workflows, it does not train the model
Running Clay's entire eval backlog	One (or a few) representative evals prove the loop
Partner-account deployment	Not needed to prove the eval loop; sandbox only
Continental-scale builds / Trainium-HyperPod orchestration	Exceeds a 2-day sandbox
Full production observability + warehouse stack (Snowflake/Iceberg/Athena, Managed Grafana)	Represent with lightweight tooling for the slice

MVP User Journeys

Journey 1: Evaluate an agentic run on a known task

1. A developer selects a task (e.g., ADB eval) and provides its input data and synthetic prompts.
2. The LLM harness runs, calling geospatial MCP tools to perform the task.
3. The observability sandbox captures the agent's traces and summarizes them.
4. The scorer rates the workflow (from traces) and the output (vs. rubric/examples).
5. The sandbox emits an evaluation report.

Outcome: A trustworthy, inspectable judgment of how the agent performed—not just a final artifact.

Limitation vs Full Vision: One task, bounded input, representative rubric; not Clay's full eval catalog.

(Stretch) Journey 2: Robustness via sensitivity analysis

1. Re-run the task while varying inputs, prompts, tool selection, or model.

2. The sandbox compares traces and scores across variants.
3. The report shows how stable the workflow and the evaluation are.

Outcome: Shows whether the agent's performance and the eval itself are robust.

Why this is a stretch: Requires the core loop solid first, plus multiple runs and comparison tooling.

MVP Definition of Done

- ☐ A chosen task runs end to end: harness → MCP tools → trace capture → summarize → score → report.
- ☐ Scoring covers both workflow and output, against a rubric or example references.
- ☐ Tracing tool and model demonstrated as swappable.
- ☐ Recorded-trace fallback verified.
- ☐ Code committed to an open-source repo the Beneficiary can access after the event.

Risks and Dependencies

Key Risks

Risk	Likelihood	Impact	Mitigation
No clear reference to score against for the chosen task	Medium	High	Confirmed sources: ADB rubric + synthetic prompts; example captions + retrieval eval set; GRW, USGS or OSM as retrieval answer keys. Secure these before the event
Live agent flaky in a 2-day build	Medium	High	Recorded-trace replay through the scoring sandbox
MCP server coverage/maturity gaps	Medium	Medium	Use Clay's free/priority shortlist from the 99; RODA MCP server as a known-good tool
NAIP Requester-Pays cost in the sandbox	Medium	Medium	Pre-stage a small NAIP sample; confirm cost handling
Scope creep across all three tasks	Medium	Medium	One task fully working first; others are stretch
Caption rubric weights not yet set	High	Medium	Report each criterion score on its own; apply weights once Clay sets them
Auxiliary caption datasets have unclear licenses or few human labels	Medium	Medium	Check each license before use; score only against human-labeled subsets (e.g., in RSCC)

External Dependencies

- **Input imagery**: NAIP on AWS — https://registry.opendata.aws/naip/ (Requester-Pays, us-west-2).
- **Sample vector data**: `s3://vantor-opendata`.
- **Embeddings**: LGND embeddings (Clay) — https://source.coop/clay/lgnd-embeddings.
- **Geospatial MCP tool**: RODA MCP server (AWS, Apache-2.0) — https://aws.amazon.com/blogs/opensource/introducing-mcp-server-for-registry-of-open-data-on-aws/ — plus Clay's free/priority shortlist from Geo-MCP servers.
    - geoMCP
- **ADB task**: input data, harness(es), scoring rubric (from Clay).
- **Caption task**: example captions + retrieval evaluation set (from Clay); five-criterion caption rubric (weights to be decided).
- **Retrieval answer keys** (suggested by ADB):
    - Global Renewables Watch v1.0, solar and wind worldwide, MIT license: https://github.com/microsoft/global-renewables-watch/releases/tag/v1.0
    - USGS USPVDB and USWTDB, US solar and wind, public domain: https://eerscmap.usgs.gov/uspvdb/ and https://eerscmap.usgs.gov/uswtdb/
    - OpenStreetMap: https://www.openstreetmap.org
- **Auxiliary caption datasets** (optional; check each license before use):
    - Change tasks: LEVIR-CC; Change-Agent, which adds change masks; SECOND-CC; RSCC, larger, but only a small subset may be human labeled.
    - Fixed tasks: VRSBench; RSICD_optimal.
- Empty AWS sandbox; open-source repo created at kickoff.

Technical Environment Document: Clay Earth Dataset Foundry

Hackathon Approach and Expectations

Read this first. It sets expectations for how to use the rest of this document.

- **Build an evaluation sandbox, not a model.** The goal is to **observe, summarize, and score agentic geospatial workflows**. The agent runs a known task by calling geospatial tools via MCP; you capture its traces and score the run. Spend your time on the observe-summarize-score loop.
- **The tasks, pick one or more.** Teams should make progress on one or more of the the tasks below; the **ADB agentic eval** is the recommended starting point.
- **Keep the model and the tracing tool swappable.** Use Amazon Bedrock for the agent, behind an abstraction. The team **recommends** a tracing/observability tool but must keep it swappable—nothing hard-wired.
- **Greenfield, self-contained, migration-aware.** Build in an empty sandbox that mirrors the setup without integrating into Clay's systems; choose shapes that map cleanly back to Clay's account.
- **Data: public or synthetic only.** NAIP imagery, Vantor open data, LGND embeddings, openly licensed reference data (GRW, USGS, OSM), and synthetic prompts/captions. Mind NAIP's Requester-Pays billing.
- **Reliability: record traces.** Capture traces so a flaky live agent can be replaced by replaying recorded traces through the scoring sandbox for the demo.

Project Technical Summary

- **Project Name**: Clay Earth Dataset Foundry (MVP — agentic-workflow evaluation sandbox)
- **Primary Runtime Environment**: Cloud
- **Cloud Provider**: AWS
- **Target Deployment Model**: Flexible (serverless / containers / K8s), single sandbox account.
- **Customer Team Experience**: Deep geospatial-ML practice—open Earth foundation model, 15.2B published embeddings (Apache-2.0), an existing evaluation harness, continental-scale inference; existing use of S3, SageMaker, Bedrock, Snowflake; maintains a catalog of ~99 MCP servers.

The Tasks (input data & scoring)

Task	Agent does	Input data (public/synthetic)	Score against
ADB agentic eval *(start here)*	Performs an ADB-style AI/dataset-build task under test	Input dataset + harness(es) under test + **synthetic prompts** (teams bring)	**Scoring rubric**; output is the **traces** gathered from observing the system

Task	Agent does	Input data (public/synthetic)	Score against
Caption generation (Clay v2)	Imagery → training captions for fixed (one date) and change (two dates) tasks, each covering position, counts and relationality; enable retrieval (e.g., "find railroads")	**NAIP** high-res imagery + **sample Vantor data** (`s3://vantor-opendata`); optional auxiliary caption datasets	**Five-criterion caption rubric** + **example captions** (Clay has them) + retrieval **evaluation datasets**; GRW, USGS or OSM as retrieval answer keys

> **Getting started (Day 1):** Start with the **ADB agentic eval**. Stand up the LLM harness on Bedrock, wire in the **RODA MCP server** (`uvx awslabs.roda-mcp-server@latest`) plus one or two from Clay's priority MCP shortlist, and get a single trace captured end to end before adding scoring. Prove the observe → summarize → score loop on one task before touching a second.

Caption Track: Tasks, Rubric and Answer Key

Task types

Task type	Input	What the caption must get right
Fixed	One image of a place at one date	Position, counts and relationality (how objects sit relative to each other) in the image
Change	Two images of the same place at different dates	Changes in position, counts and relationality between the two dates

Rubric and how to measure it

Five criteria. Weighting is still to be decided, so score and report each criterion on its own and apply weights later.

Criterion	What it checks	How to measure
Retrieval quality	A caption can be found from a text query alone, with no image	Embed captions and queries with any off-the-shelf text model; record how often the right caption is in the top results
Grounding (most important)	The caption matches what is on the ground: objects, attributes and counts	Compare caption claims with reference labels (GRW, USGS, OSM, Vantor vectors or human-labeled datasets)
Fidelity	The caption represents the image accurately overall	An LLM judge compares caption and image and gives a score

Criterion	What it checks	How to measure
Cost	Throughput and time to output across VLMs	Same images and prompt for each VLM; record GPU time and number or size of GPUs, not token cost
Label preparation time	How long labels take to prepare	Record the time taken to prepare each label set

Retrieval answer key (ADB suggestion)

Train on one region of Global Renewables Watch v1.0 or USGS USPVDB/USWTDB, then test on another region that already has an LGND collection, using GRW, USGS or OSM there as the answer key.

Programming Languages

Required Languages

Language	Version	Purpose	Rationale
Python	3.12+	Agent/harness, MCP wiring, trace capture/scoring, geospatial processing	Geospatial + agent/MCP ecosystems live in Python; matches Clay
TypeScript / JavaScript	5.x / Node 20+	Thin UI; MCP servers if authored in TS	Standard thin front end; strong MCP support

Permitted Languages

SQL (lightweight trace/metadata store), Bash (sandbox scripting).

Frameworks and Libraries

Required / Expected

Framework/Library	Domain	Rationale
An agent framework (kept swappable)	LLM harness: plan → tool-call → task	The agentic workflow under evaluation; framework not prescribed
MCP SDK + geospatial MCP server(s)	Expose geospatial tools to the agent	RODA MCP server + Clay's free/priority shortlist from 99
A tracing/observability tool (team recommends, swappable)	Capture + summarize agent traces	Confirmed: team chooses, must stay swappable (an OpenTelemetry-based LLM trace approach is a reasonable default)

Framework/Library	Domain	Rationale
Geospatial stack (rasterio / GDAL / xarray / pystac)	Imagery handling, STAC assets	Standard toolkit for NAIP/public archives
AWS SDK for Python (boto3)	S3, Bedrock access	Native AWS integration

Preferred Libraries

Library	Purpose	Use When
Clay / LGND embeddings (Apache-2.0)	Imagery embeddings for captioning/retrieval	Reuse rather than train
scikit-learn / numpy	Scoring metrics (output vs. reference)	Scoring the task output
An off-the-shelf text embedding model	Caption retrieval-quality score (text query to caption, no image)	Scoring the caption task
FastAPI	Thin API behind the UI	Launching runs, serving reports

Models and AI Services

- **Provider**: Amazon Bedrock for the agent's reasoning, behind an abstraction so the model is easy to swap.
- **Embeddings**: Clay / LGND embeddings for the caption/retrieval task.
- **Caption VLMs**: Run the caption task on more than one VLM with the same images and prompt, so cost can be compared in GPU time and GPU count and size.
- **MCP**: The agent reaches geospatial capabilities through MCP servers. Start with the **RODA MCP server** (AWS, Apache-2.0) and Clay's **free/priority shortlist** from its 99 servers.

Observability and Evaluation (the heart of the build)

- **Capture**: record every agent step—tool calls, arguments, sequence, failures/retries, latency, cost. For caption runs, record cost as GPU time and GPU count and size rather than tokens.
- **Summarize**: turn raw traces into a readable summary of what the agent did and where it struggled (Clay's explicit ask: "summarize traces from agentic activity").
- **Score — two dimensions**:
  - **Workflow (from traces)**: tool-call correctness and sequencing, error recovery, steps/cost.
  - **Output**: quality vs. the task's reference—**rubric** (ADB) or **example captions + retrieval eval set** (captions), scored on the five-criterion **caption rubric**.
- **Report**: a per-run evaluation report combining the summary and the scores.
- **Swappable tooling**: the tracing/observability tool is the team's recommendation but must sit behind an abstraction; same for the model.

- **Fallback**: persist traces so recorded runs can be replayed through scoring if the live agent misbehaves.

Cloud Environment

Cloud Provider

- **Primary Provider**: AWS
- **Account Structure**: Single empty hackathon sandbox account (greenfield). No connection to Clay's production account.
- **Regions**: Note **NAIP is in us-west-2 and Requester-Pays**—account for region and billing (pre-stage a small sample if helpful).

Confirmed Data Sources

Source	What	Access notes
NAIP on AWS	High-res (0.3–1 m) 4-band US aerial imagery	`s3://naip-analytic` (MRF), `naip-source` (GeoTIFF), `naip-visualization` (RGB COG); **Requester-Pays**, **us-west-2**
Vantor open data	Sample vector data for captioning	`s3://vantor-opendata`
LGND embeddings	Clay/LGND embeddings	https://source.coop/clay/lgnd-embeddings
RODA MCP server	AWS Apache-2.0 MCP server for Registry of Open Data discovery/sampling	`uvx awslabs.roda-mcp-server@latest`
MCP server catalog	Clay's ~99 MCP servers	Clay to share **free + priority** shortlist
Global Renewables Watch v1.0	Solar and wind sites worldwide; retrieval answer key	MIT license; https://github.com/microsoft/global-renewables-watch/releases/tag/v1.0
USGS USPVDB / USWTDB	US solar and wind sites; retrieval answer key	Public domain; https://eerscmap.usgs.gov/uspvdb/ and https://eerscmap.usgs.gov/uswtdb/
OpenStreetMap	Map features; retrieval answer key	ODbL license; https://www.openstreetmap.org
LGND Geo MCP server	Provides access to easy search of Earth data for LLMs and LLM tools.	This is a commercial product. You will be whitelisted to use it for the purposes of the hackathon. https://lgnd.ai/geo

Source	What	Access notes
SkyScript	“A Large and Semantically Diverse Vision-Language Dataset for Remote Sensing"	https://github.com/wangzhechengg/skyscript
MegaEarth	MegaEarth pairs Sentinel-2 windows with OpenStreetMap labels for 485 classes, grouped into families and split by geography, testing whether image features transfer to unseen categories.	https://huggingface.co/datasets/Mead0w1ark/megaearth-osm-sentinel2

Auxiliary Caption Datasets (optional)

Check each license before use.

Dataset	Task type	Notes
LEVIR-CC	Change	Change captioning and detection
Change-Agent	Change	Adds change masks
SECOND-CC	Change	Change captioning
RSCC	Change	Larger set; only a small subset may be human labeled
VRSBench	Fixed	Single-image descriptions
RSICD_optimal	Fixed	Single-image captions

Service Disallow List

Service	Reason	Alternative
SageMaker HyperPod / large Trainium clusters	Overkill for a sandbox eval slice	Modest instance / managed endpoint / mocked inference
Multi-account / cross-account deployment	Out of MVP scope	Single sandbox account
Full Snowflake/Iceberg/Athena warehouse	Heavy; not needed to prove the loop	Lightweight trace/metadata store
Any customer-private/sensitive data	Sandbox is public/synthetic only	NAIP, Vantor, LGND embeddings, open reference data (GRW, USGS, OSM), synthetic prompts/captions

Preferred Technologies and Patterns

Pattern	When to Use	When Not to Use
Observe → summarize → score the agent	Default: every run is traced, summarized, and scored	Never evaluate on final output alone with no trace
Model + tracing tool behind abstractions	Keep both swappable	Never hard-wire a specific model or tracing vendor
Recorded-trace replay	Demo reliability and offline scoring	—
Read data at the source via MCP	Stream NAIP/Vantor through MCP tools	When a tiny pre-staged sample is faster/cheaper (Requester-Pays)
Greenfield, migration-aware	Self-contained sandbox, shapes that map to Clay's account	When a shortcut makes migration harder

Stretch Goals (if the team moves fast)

- Evaluate a **second (or third) task** to show the loop generalizes.
- **Multi-agent** (planner/executor/checker) traces.
- **Dynamic MCP tool discovery** across Clay's 99-server catalog.
- A richer **trace-summary dashboard**.
- **Sensitivity analysis** — vary inputs, prompts, tool selection, or model and measure how much the output and the trace/scores change (robustness of the workflow and the eval).